

Manual de Usuario

Sistema de Alquiler de Bicicletas — Trabajo Práctico de Programación Concurrente (75.59), FIUBA, 1er cuatrimestre 2026.

Integrantes: Guido Peirano (98187) · Tomás Goncalves Rei (111405) · Paul García López (112363).

Este manual explica cómo poner en marcha el sistema, qué comando correr en cada proceso, y el paso a paso de **todos los flujos** que pide la cátedra — incluidos los flujos de falla y cómo el sistema los trata. Está pensado como guía práctica para la demo: si en la defensa piden un flujo puntual, acá está cómo ejecutarlo y qué observar.

1. Qué es el sistema

Es un sistema distribuido para alquilar y devolver bicicletas en una ciudad. Está compuesto por **tres tipos de proceso independientes** que se comunican por sockets, cada uno en su propia terminal:

- **estacion** (varias instancias): maneja sus slots, asegura y libera bicis, coordina el alquiler con la pasarela y mantiene el anillo de elección de líder. Una estación es el **líder** y lleva el registro autoritativo de alquileres.
- **pasarela** (una instancia): simula el procesador de pagos; hace pre-autorizaciones y cobros.
- **usuario** (varias instancias): cliente de consola para alquilar, devolver y consultar disponibilidad.

Usa dos herramientas de concurrencia distribuida: **Two-Phase Commit (2PC)** para la atomicidad del alquiler, y **elección de líder por algoritmo Ring** para tolerar la caída del líder. Hecho en Rust, con el modelo de actores (actix), sin bloques `unsafe`.

Qué se puede hacer. Alquilar una bici · devolverla (con cobro por tiempo de uso) · consultar estaciones cercanas con disponibilidad · y simular fallas (cortes de red de estaciones/pasarela, caída del líder, usuario sin señal) para ver la resiliencia del sistema.

2. Puesta en marcha

2.1 Requisitos y compilación

Sistema operativo Unix/Linux (o WSL en Windows) con Rust y Cargo instalados. Desde la raíz del repositorio, compilar todo el workspace una sola vez:

```
cargo build --release
```

2.2 El archivo de configuración (estaciones.json)

Todos los procesos arrancan con el mismo archivo de topología. El de ejemplo en la raíz es:

```
{
  "estaciones": [
    { "id": 1, "puerto": 8001, "ubicacion": [-34.60, -58.40] },
    { "id": 2, "puerto": 8002, "ubicacion": [-34.51, -58.40] },
    { "id": 3, "puerto": 8003, "ubicacion": [-34.42, -58.40] }
  ],
  "pasarela": { "puerto": 9000 },
  "tarifa": { "base": 50.0, "por_minuto": 10.0 },
  "lider": 1
}
```

- **estaciones:** id, puerto (TCP+UDP) y ubicación (lat, lon) de cada estación. El anillo de elección se ordena por id.
- **pasarela:** puerto donde escucha la pasarela.
- **tarifa:** el cobro es $\text{base} + \text{por_minuto} * \text{minutos_de_uso}$ (acá \$50 + \$10/min).
- **lider:** estación que arranca como líder (luego lo decide el algoritmo Ring).

Distancias entre estaciones. Las estaciones están sobre la misma longitud y separadas ~10 km entre sí: $E1 \leftrightarrow E2 \approx 10 \text{ km}$, $E2 \leftrightarrow E3 \approx 10 \text{ km}$, $E1 \leftrightarrow E3 \approx 20 \text{ km}$. Esto hace fácil de verificar la consulta por radio (ver Flujo F3).

2.3 Levantar el sistema (orden recomendado)

Abrir una terminal por proceso. Orden: **primero la pasarela, después las estaciones, y al final los usuarios.**

```
# 1) Pasarela de pagos
cargo run --release --bin pasarela -- --puerto 9000 --config estaciones.json

# 2) Estaciones (una por terminal)
cargo run --release --bin estacion -- --id 1 --puerto 8001 --config estaciones.json
cargo run --release --bin estacion -- --id 2 --puerto 8002 --config estaciones.json
cargo run --release --bin estacion -- --id 3 --puerto 8003 --config estaciones.json

# 3) Usuarios (una por terminal)
cargo run --release --bin usuario -- --id alice --config estaciones.json
cargo run --release --bin usuario -- --id bob --config estaciones.json
```

Binario	Argumentos
pasarela	--puerto <p> --config <archivo> [--estado <ruta.json>]
estacion	--id <n> --puerto <p> --config <archivo> [--estado <ruta.json>]
usuario	--id <nombre> --config <archivo>

--estado <ruta> (opcional, en estación y pasarela) activa la **persistencia**: el estado sobrevive a reinicios, con escritura atómica. Útil para los flujos de reinicio (ver F10).

2.4 Qué se ve al arrancar (confirmar que levantó bien)

Pasarela:

```
[pasarela] config cargada: puerto=9000, tarifa base=50, por_minuto=10
[pasarela] escuchando en 127.0.0.1:9000 (TCP); a la espera (ctrl-c para salir)
```

Estación (ejemplo, id 1):

```
[estacion 1] config cargada: puerto=8001, ubicacion=(-34.6, -58.4), pasarela_puerto=9000, estaciones=3
[estacion 1] escuchando en 127.0.0.1:8001 (TCP+UDP); 10 slots; a la espera (ctrl-c para salir)
```

Usuario: lista las estaciones conocidas y muestra la línea de ayuda con los comandos disponibles.

2.5 Estado inicial de las estaciones

Cada estación arranca con **10 slots**. La primera mitad viene con una bici y el resto vacío:

Estación	Slots con bici (0–4)	Slots vacíos (5–9)	Ids de bici
1	0, 1, 2, 3, 4	5, 6, 7, 8, 9	100, 101, 102, 103, 104
2	0, 1, 2, 3, 4	5, 6, 7, 8, 9	200, 201, 202, 203, 204
3	0, 1, 2, 3, 4	5, 6, 7, 8, 9	300, 301, 302, 303, 304

Regla práctica. Se **alquila** desde un slot que tiene bici (0–4) y se **devuelve** en un slot vacío (5–9). El id de la bici se deriva de la estación: estación N, slot i → bici $N \cdot 100 + i$.

3. Referencia de comandos por proceso

3.1 Usuario (REPL)

Comando	Qué hace
alquilar <estacion> <slot>	Pide alquilar la bici del slot indicado en esa estación.
devolver <estacion> <slot>	Devuelve la bici que tenés en un slot vacío de esa estación.
consultar <lat> <lon> <radio_km>	Lista estaciones con bici dentro del radio (requiere conectividad).
desconectar	Simula perder la señal: pasa a modo SoloLocal.
conectar	Recupera la señal: vuelve a modo Conectado.
estado	Muestra si tenés bici y tu modo de conectividad.
ayuda	Reimprime la lista de comandos.
salir / exit	Cierra el cliente.

3.2 Estación (consola del proceso)

Comando	Qué hace
desconectar	Simula que la estación pierde la red (descarta todo lo que entra y sale).
conectar	Restablece la red de la estación.
logs	Activa/desactiva (toggle) la traza del gossip UDP de estado.

3.3 Pasarela (consola del proceso)

Comando	Qué hace
desconectar	Simula que la pasarela se cae / queda inalcanzable.
conectar	Restablece la pasarela.

3.4 Glosario de respuestas del usuario

- AlquilerConfirmado { rental_id, bici_id, preauth_id } — alquiler OK. Si preauth_id es Some(..) fue con pasarela; si es None, salió en **modo offline** (Caso E).
- AlquilerRechazado { motivo } — no se pudo (slot vacío, o la pasarela votó No por tarjeta inválida).
- DevolucionAceptada { bici_id } — la bici quedó asegurada; el cobro y el cierre corren en segundo plano.
- DevolucionRechazada { motivo } — el slot de destino estaba ocupado u otro motivo.

4. Catálogo de flujos

Cada flujo indica: **objetivo**, **precondición**, **pasos** (con la terminal en la que se ejecuta) y **qué observar**. Conviene activar `logs` en una estación para ver el 2PC y el gossip durante la demo.

F1 — Alquiler exitoso (CU1 · 2PC)

Objetivo: mostrar el alquiler completo con atomicidad (Two-Phase Commit). **Precondición:** sistema levantado; la pasarela conectada.

1. **(usuario alice)** `alquilar 1 0`
2. **(estación 1)** observar en los logs el 2PC: Prepare al Slot y a la Pasarela → ambos Voto(Yes) → Commit.
3. **(usuario alice)** `estado` → debe figurar `ConBici`.

Qué observar. El usuario recibe `AlquilerConfirmado { bici_id: BiciId(100), preauth_id: Some(..) }`. La bici sale del slot 0 de la estación 1. Tras confirmar, la estación reporta el alquiler al líder de forma asíncrona (el usuario no espera por eso).

F2 — Devolución exitosa con cobro (CU2)

Objetivo: devolver en una estación distinta y ver el cobro por tiempo de uso. **Precondición:** venir de F1 (alice tiene la bici 100).

1. **(usuario alice)** esperar unos segundos (para que el cobro por minuto sea visible) y luego `devolver 2 5`.
2. **(estación 2)** observar: asegura la bici, le pide al líder los datos del alquiler (`DatosParaCobro`), cobra a la pasarela y notifica el cierre al líder y a la estación de origen (la 1).
3. **(usuario alice)** `estado` → debe figurar `SinBici`.

Qué observar. El usuario recibe `DevolucionAceptada` y puede irse; el cobro y el cierre corren en segundo plano. En los logs de la estación 2 se ve el monto = base + por_minuto * minutos (p. ej. \$50 + \$10/min). La bici queda asegurada en el slot 5 de la estación 2. Importante: **cobra la estación destino, no el líder** (decisión de diseño tras la corrección de cátedra).

F3 — Consulta de disponibilidad por radio (CU3)

Objetivo: ver estaciones con bici cerca de un punto; muestra el descubrimiento del líder y el filtro por proximidad. **Precondición:** usuario en modo Conectado.

1. **(usuario bob)** `consultar -34.60 -58.40 5` → solo la **estación 1** (radio 5 km).
2. **(usuario bob)** `consultar -34.60 -58.40 15` → estaciones **1 y 2** (radio 15 km).
3. **(usuario bob)** `consultar -34.60 -58.40 25` → las **tres** estaciones (radio 25 km).

Qué observar. Por cada estación dentro del radio se imprime: `estación N en (lat, lon): X bicis, Y slots libres`. El cliente primero descubre al líder preguntando a las estaciones de la config, y le consulta directo a él (el líder arma la respuesta con el gossip UDP que recibe de todas).

F4 — Alquiler rechazado por slot vacío (2PC · Voto No del Slot)

Objetivo: mostrar un Voto No legítimo y el abort del 2PC. **Precondición:** sistema levantado.

1. **(usuario alice)** `alquilar 1 7` (el slot 7 está vacío, no tiene bici).

Qué observar. El usuario recibe `AlquilerRechazado { motivo: .. }`. El Slot vota No porque no tiene bici; el coordinador aborta y no se toca la pasarela. El estado del usuario sigue en `SinBici`.

F5 — Devolución rechazada por slot ocupado

Objetivo: mostrar que no se puede devolver en un slot que ya tiene bici. **Precondición:** alice con una bici (venir de F1).

1. **(usuario alice)** `devolver 2 0` (el slot 0 de la estación 2 ya tiene bici).

Qué observar. El usuario recibe `DevolucionRechazada { motivo: .. }` y conserva la bici. Reintentar en un slot vacío (5–9) la acepta.

F6 — Estación sin pasarela: alquiler offline y regularización (CU6 · Caso E)

Objetivo: demostrar que el usuario puede retirar la bici aunque la estación no alcance a la pasarela, y que el pago se regulariza al reconectar. **Precondición:** sistema levantado, alice sin bici.

1. **(pasarela)** `desconectar` (simula la pasarela caída).
2. **(usuario alice)** `alquilar 1 0` → la bici se entrega igual.
3. **(usuario alice)** `estado` → `ConBici`. Notar que en la respuesta `preauth_id` es `None`.
4. **(estación 1)** observar: registra el alquiler con `preauth_id = None`, persiste un `PagoPendiente` y **no** reporta al líder.
5. **(pasarela)** `conectar`.
6. **(estación 1)** observar: regulariza el `PagoPendiente` (obtiene la pre-autorización real y recién ahí reporta el alquiler al líder).
7. **(usuario alice)** `devolver 2 5` → recién acá ocurre el cobro, por el flujo normal de devolución.

Qué observar. El alquiler offline confirma con `preauth_id: None`. Es una decisión consciente: priorizamos la disponibilidad (que el usuario saque la bici sin banco) sobre la atomicidad estricta. Regla clave: **ningún alquiler llega al líder sin pre-autorización**, por eso el reporte se difiere hasta reconectar. El cobro nunca ocurre en el alquiler offline; ocurre al devolver.

F7 — Devolución con pasarela caída: el cobro se reintenta

Objetivo: mostrar que la devolución no se cierra en \$0 si la pasarela está caída; reintenta hasta que vuelve. **Precondición:** alice con una bici (de un alquiler normal, con `preauth`).

1. **(pasarela)** `desconectar`.
2. **(usuario alice)** `devolver 2 5` → recibe `DevolucionAceptada` (la bici queda guardada).
3. **(estación 2)** observar: intenta cobrar y falla; reintenta con backoff sin cerrar el alquiler.
4. **(pasarela)** `conectar`.
5. **(estación 2)** observar: el reintento prospera, se cobra y se cierra el alquiler con el origen.

Qué observar. El usuario ya se fue con `DevolucionAceptada`; el cobro queda pendiente y se completa cuando la pasarela vuelve. El cobro es idempotente: si se reintenta sobre una `preauth` ya cobrada, devuelve el mismo monto (no cobra dos veces).

F8 — Usuario sin señal: modo SoloLocal (CU5)

Objetivo: distinguir el acceso global (consulta) del local (alquilar/devolver). **Precondición:** sistema levantado.

1. **(usuario bob)** `desconectar` → pasa a modo SoloLocal.
2. **(usuario bob)** `consultar -34.60 -58.40 25` → responde que la consulta no está disponible sin señal.
3. **(usuario bob)** `alquilar 2 0` → **sí funciona** (es una conexión local a una estación cercana).
4. **(usuario bob)** `conectar` → vuelve a modo Conectado; la consulta vuelve a estar disponible.

Qué observar. En SoloLocal, `consultar` imprime «sin señal (SoloLocal): la consulta de disponibilidad no está disponible», pero `alquilar` y `devolver` contra una estación cercana siguen andando. Es exactamente lo que pide el enunciado: sin señal, igual se puede retirar una bici.

F9 — Caída del líder: re-elección y reconstrucción (CU4)

Objetivo: mostrar la elección de líder por Ring y que el sistema sigue operando. **Precondición:** las 3 estaciones levantadas (el líder inicial es la estación 1 por config; gana el id mayor en una elección).

1. **(estación líder)** `desconectar` (simula la caída del líder).
2. **(otra estación)** `observar`: timeout esperando al líder → inicia `Election` → da la vuelta al anillo → `Coordinator(nuevo_lider, term+1)`.
3. **(usuario alicé)** durante la elección, una `consultar` responde que hay una elección en curso (reintentar). En cambio, `alquilar/devolver` locales siguen funcionando.
4. **(estación que ganó)** `observar`: pide a todas `SolicitarAlquileresAbiertos` y reconstruye el registro.
5. **(estación caída)** `conectar` → se reincorpora como follower (no vuelve como líder).

Qué observar. El registro de alquileres se reconstruye a partir de las copias locales de cada estación, así que no se pierde nada. Mientras dura la elección (unos segundos), lo que requiere al líder se encola y se despacha al nuevo líder. El `term` mayor evita que haya dos líderes.

F10 — Reinicio del ex-líder: anti split-brain

Objetivo: mostrar que una estación que se reinicia no vuelve creyéndose líder. **Precondición:** conviene haber arrancado las estaciones con `--estado` para que el estado sobreviva.

1. Provocar un cambio de líder (como en F9) para que el líder ya no sea la estación 1.
2. **(estación 1)** cortar el proceso con Ctrl-C y volver a lanzarlo con el mismo comando.
3. **(estación 1)** `observar` al arrancar: pregunta a sus vecinas `QuienEsLider` y adopta el líder/`term` vigente, reincorporándose como **follower**.

Qué observar. Aunque la config la designa líder, al reiniciar consulta a las vecinas y no se autoproclama. Así se evita el split-brain (dos líderes) en el caso de reinicio.

Limitación conocida (saber explicarla). Si una estación se aísla por una partición de red SIN reiniciarse y se autoelige, al sanar la partición pueden quedar momentáneamente dos líderes hasta que una nueva elección propague un `term` mayor. Cubrimos el reinicio; la reconciliación de dos líderes activos queda como mejora futura.

F11 — Líder colgado y bici huérfana (avanzado / mención)

- **Líder colgado (no caído):** un proceso puede aceptar la conexión TCP pero no responder. Lo detectamos con un timeout de lectura (5 s) y un ACK en el Ring, que disparan la re-elección igual que una caída. Se puede aproximar con `desconectar` en el líder (F9).

- **Bici huérfana:** si una bici llega a un slot pero el líder no encuentra su alquiler (evento perdido o reconstrucción incompleta), la estación destino busca al dueño entre las estaciones; si nadie la reconoce, la bici queda disponible y se audita. Es difícil de forzar a mano de forma confiable; se explica como caso contemplado.

F12 — Forzar un rechazo por tarjeta inválida (avanzado)

Por defecto **no se puede** disparar un rechazo por tarjeta desde el usuario: la tarjeta del cliente está hardcodeda como válida, así que la pasarela siempre vota Yes (por eso el rechazo de F4 lo hacemos con un slot vacío). Para mostrar el camino «la pasarela vota No por tarjeta», hay que editar el cliente.

Dónde está la validación (pasarela): la función `tarjeta_valida` exige número no vacío y CVV de 3 dígitos.

```
// pasarela/src/procesador_pagos.rs (~línea 260)
/// Validación de tarjeta (mock): exige número no vacío y CVV de 3 dígitos.
fn tarjeta_valida(tarjeta: &DatosTarjeta) -> bool {
    !tarjeta.numero.is_empty() && tarjeta.cvv.len() == 3
}
```

Dónde está la tarjeta del usuario: en el constructor del cliente.

```
// usuario/src/usuario.rs (~línea 39)
tarjeta: DatosTarjeta {
    numero: "4111111111111111".to_string(),
    titular: "Titular Demo".to_string(),
    vencimiento: "12/29".to_string(),
    cvv: "123".to_string(), // <-- cambiar a "12" (o numero a "") para invalidarla
},
```

Pasos para la demo:

1. Editar `usuario/src/usuario.rs`: poner `cvv: "12"` (dos dígitos) o `numero: ""`.
2. Recompilar: `cargo build --release` y relanzar ese usuario.
3. Con la **pasarela conectada**, ejecutar `alquilar 1 0`.
4. Revertir el cambio al terminar la demo.

Qué observar. El usuario recibe `AlquilerRechazado { motivo: "tarjeta inválida" }`. En la pasarela se ve `PreparePreauth ...: VOTO NO (tarjeta inválida)`. El coordinador aborta el 2PC y el Slot no pierde la bici. Ojo: con la **pasarela desconectada** el resultado es distinto — no es un rechazo sino un alquiler offline (F6), porque un no-responde se trata como inalcanzable, no como Voto No.

5. Mapa flujo ↔ requerimiento de la cátedra

Índice rápido para la defensa: si piden validar un requerimiento puntual, este es el flujo que lo demuestra.

Requerimiento / herramienta	Flujo	Qué demuestra
Alquiler atómico (2PC)	F1, F4	Commit con ambos votos; abort ante Voto No
Devolución y cobro por tiempo	F2	Cobro = base + por_minuto; lo hace la estación destino
Minimizar tráfico / cercanía (consulta)	F3	Filtro por proximidad y descubrimiento del líder
Funciona desconectado: usuario sin señal	F8	SoloLocal: alquila local, no consulta global
Funciona desconectado: estación sin pagos	F6	Alquiler offline + PagoPendiente + regularización
Resiliencia de la devolución	F7	Reintento del cobro idempotente; no cierra en \$0
Elección de líder (Ring) / caída del líder	F9, F10	Re-elección, reconstrucción del registro, anti split-brain
Idempotencia / manejo de fallas del 2PC	F4, F6, F7	Voto No, offline, reintentos seguros
Rechazo de pago (Voto No de la pasarela)	F12	Tarjeta inválida → abort del 2PC

6. Tips y resolución de problemas

- **«no pude contactar ninguna estación de la config»** al consultar → no hay estaciones levantadas, o están todas desconectadas. Verificá las terminales de las estaciones.
- **«hay una elección en curso ... probá de nuevo en unos segundos»** → hay una re-elección activa; reintentar la consulta.
- **«ya tenés la bici ...»** al alquilar → un usuario sostiene una sola bici; primero devolver.
- Recordá: se **alquila** de un slot 0–4 (con bici) y se **devuelve** en un slot 5–9 (vacío).
- Para los flujos de reinicio (F10), arrancá estación/pasarela con `--estado <ruta.json>` para que el estado persista.
- Activá `logs` en una estación para que se vean el 2PC, el gossip UDP y el reconocimiento de líder durante la demo.
- Si algo falla en vivo, explicalo desde la teoría (causa + cómo se rectifica): la consigna lo premia.